

Tugas 3

Struktur Data

Advanced Sorting Algorithms



Dosen Pengampu :

M. Chandra Cahyo Utomo, S. Kom., M. Kom.

NIP. 199202202019031000

Disusun Oleh Kelompok 2 :

Cindy Aulia Renita	11241023
Fauzan Fayzul Haq	11241028
Thisya Darmala	11241083
Zharif Aziz Zulkarnain Widodo	11231092

10 Oktober 2025

Dasar Teori

Merge Sort

Merge sort merupakan algoritma pengurutan yang juga menggunakan pendekatan *divide and conquer*. Algoritma ini bekerja dengan membagi array menjadi dua sub-array yang lebih kecil. Kemudian, *sub-array* ini diurutkan secara terpisah dan digabungkan kembali untuk membentuk array yang terurut. Proses penggabungan dilakukan dengan membandingkan elemen-elemen dari kedua *sub-array* dan menempatkannya ke dalam array yang terurut.

Cara kerja *Merge Sort*:

- 1) Membagi array menjadi dua *sub-array* yang lebih kecil hingga setiap *sub-array* hanya berisi satu elemen.
- 2) *Sub-array* yang hanya berisi satu elemen dianggap sudah terurut. Kemudian, *sub-array* ini digabungkan kembali untuk membentuk array yang terurut.
- 3) Proses penggabungan dilakukan dengan membandingkan elemen-elemen dari kedua *sub-array* dan menempatkannya ke dalam array yang terurut

Quick Sort

Quick sort merupakan algoritma pengurutan yang menggunakan pendekatan *divide and conquer*. Dalam *quick sort*, array dibagi menjadi dua partisi berdasarkan *pivot*. *Pivot* adalah elemen yang dipilih dari array. Semua elemen yang lebih kecil dari *pivot* ditempatkan di sebelah kiri *pivot*, dan semua elemen yang lebih besar dari *pivot* ditempatkan di sebelah kanan *pivot*.

Cara kerja *Quick Sort*:

- 1) Memilih *pivot* dari array. *Pivot* dapat menjadi elemen pertama, terakhir, atau tengah dari array.
- 2) Proses ini diulang secara rekursif untuk partisi kiri dan kanan hingga semua elemen terurut.
- 3) Semua elemen yang lebih kecil dari *pivot* ditempatkan di sebelah kiri *pivot*, dan semua elemen yang lebih besar dari *pivot* ditempatkan di sebelah kanan *pivot*.

Shell Sort

Shell sort merupakan algoritma pengurutan yang bekerja dengan membagi array menjadi *sub-array* yang lebih kecil. Kemudian, *sub-array* ini diurutkan secara terpisah dan digabungkan kembali untuk membentuk array yang terurut. *Shell sort* menggunakan interval gap untuk membandingkan dan menukar elemen-elemen yang jauh.

Cara kerja *Shell Sort*:

- 
- 1) Algoritma *Shell Sort* dimulai dengan memilih gap awal, yang biasanya merupakan setengah dari dari ukuran *array*. Gap ini digunakan untuk membandingkan dan menukar elemen yang jauh.
 - 2) *Array* kemudian dibagi menjadi *sub-array* berdasarkan gap yang dipilih, dan *sub-array* ini diurutkan menggunakan insertion sort.
 - 3) Proses ini diulang dengan gap yang lebih kecil hingga akhirnya gap menjadi 1. Pada saat gap 1, *shell sort* melakukan pengurutan linear seperti *insertion sort*.

Source Code

No	Merge.java
1	<code>package Tugas_3;</code>
2	
3	<code>import java.util.Arrays;</code>
4	
5	<code>public class Merge {</code>
6	<code> public static void mergeSort(int[] arr, boolean verbose)</code>
7	<code>{</code>
8	<code> if (arr == null arr.length <= 1) return;</code>
9	<code> int mid = arr.length / 2;</code>
10	<code> int[] left = new int[mid];</code>
11	<code> int[] right = new int[arr.length - mid];</code>
12	
13	<code> for (int i = 0; i < mid; i++) {</code>
14	<code> left[i] = arr[i];</code>
15	<code> }</code>
16	<code> for (int i = mid; i < arr.length; i++) {</code>
17	<code> right[i - mid] = arr[i];</code>
18	<code> }</code>
19	<code> if (verbose) System.out.println("Split menjadi left:</code>
20	<code>" + Arrays.toString(left) + " dan right: " +</code>
21	<code>Arrays.toString(right));</code>
22	
23	<code> mergeSort(left, verbose);</code>
24	<code> mergeSort(right, verbose);</code>
25	<code> merge(arr, left, right, verbose);</code>
26	<code> }</code>
27	<code> private static void merge(int[] arr, int[] left, int[]</code>
28	<code>right, boolean verbose) {</code>
29	<code> if (verbose) System.out.println("Merging left: " +</code>
30	<code>Arrays.toString(left) + " dan right: " +</code>
31	<code>Arrays.toString(right));</code>
32	<code> int i = 0, j = 0, k = 0;</code>
33	<code> while (i < left.length && j < right.length) {</code>
34	<code> if (left[i] <= right[j]) {</code>
35	<code> arr[k++] = left[i++];</code>
36	<code> if (verbose) System.out.println("Ambil dari</code>
37	<code>left: " + left[i-1]);</code>
38	<code> } else {</code>
39	<code> arr[k++] = right[j++];</code>
40	<code> if (verbose) System.out.println("Ambil dari</code>
41	<code>right: " + right[j-1]);</code>
42	<code> }</code>
41	<code> if (verbose) System.out.println("Array merging</code>
42	<code>sementara: " + Arrays.toString(arr));</code>

43	}
44	while (i < left.length) {
45	arr[k++] = left[i++];
46	if (verbose) System.out.println("Ambil sisa left:
47	" + left[i-1]);
48	if (verbose) System.out.println("Array merging
49	sementara: " + Arrays.toString(arr));
50	}
51	while (j < right.length) {
52	arr[k++] = right[j++];
53	if (verbose) System.out.println("Ambil sisa
54	right: " + right[j-1]);
55	if (verbose) System.out.println("Array merging
56	sementara: " + Arrays.toString(arr));
57	}
58	if (verbose) System.out.println("Hasil merge: " +
59	Arrays.toString(arr));
60	}

No	Quick.java
1	package Tugas_3;
2	
3	import java.util.Arrays;
4	
5	public class Quick {
6	public static void quickSort(int[] arr, int low, int
7	high, boolean verbose) {
8	if (low < high) {
9	int pi = partition(arr, low, high, verbose);
10	if (verbose) System.out.println("Setelah
11	partition (pivot di " + pi + "): " + Arrays.toString(arr));
12	quickSort(arr, low, pi - 1, verbose);
13	quickSort(arr, pi + 1, high, verbose);
14	}
15	
16	private static int partition(int[] arr, int low, int
17	high, boolean verbose) {
18	int pivot = arr[high];
19	if (verbose) System.out.println("Pivot: " + pivot + "
20	(indeks " + high + ")");
21	int i = low - 1;
22	for (int j = low; j < high; j++) {
23	if (arr[j] <= pivot) {
24	i++;
25	if (verbose) System.out.println("Swap: " +
	arr[i] + " dengan " + arr[j] + " (indeks " + i + " dan " + j

```

26 + "));
27         int temp = arr[i];
28         arr[i] = arr[j];
29         arr[j] = temp;
30         if (verbose) System.out.println("Array
31 sementara: " + Arrays.toString(arr));
32     }
33 }
34     if (verbose) System.out.println("Swap pivot: " +
35 arr[i + 1] + " dengan " + arr[high] + " (indeks " + (i + 1)
36 + " dan " + high + "));
37     int temp = arr[i + 1];
38     arr[i + 1] = arr[high];
39     arr[high] = temp;
40     if (verbose) System.out.println("Array setelah swap
41 pivot: " + Arrays.toString(arr));
42     return i + 1;
43 }
44 }

```

No	Shell.java
1	package Tugas_3;
2	
3	import java.util.Arrays;
4	
5	public class Shell {
6	public static void shellSort(int[] arr, boolean verbose)
7	{
8	int n = arr.length;
9	for (int gap = n/2; gap > 0; gap /= 2) {
10	if (verbose) System.out.println("Gap: " + gap);
11	for (int i = gap; i < n; i++) {
12	int temp = arr[i];
13	int j;
14	for (j = i; j >= gap && arr[j - gap] > temp;
15	j -= gap) {
16	if (verbose) System.out.println("Swap: "
17	+ arr[j - gap] + " dengan " + arr[j]);
18	arr[j] = arr[j - gap];
19	if (verbose) System.out.println("Array
20	sementara: " + Arrays.toString(arr));
21	arr[j] = temp;
22	if (verbose && j != i) {
23	System.out.println("Insert " + temp + "
24	ke posisi " + j);
25	System.out.println("Array setelah insert:

```

26 " + Arrays.toString(arr));
27     }
28 }
29 }
30 }
31 }

```

No	Main.java
1	<code>package Tugas_3;</code>
2	
3	<code>import java.util.Arrays;</code>
4	<code>import java.util.Scanner;</code>
5	<code>import java.util.Random;</code>
6	
7	<code>public class Main {</code>
8	<code>public static void main(String[] args) {</code>
9	<code>Scanner scanner = new Scanner(System.in);</code>
10	<code>Random rand = new Random();</code>
11	<code>boolean running = true;</code>
12	
13	<code>System.out.print("Masukkan ukuran array untuk</code>
14	<code>pengujian individu (default 10): ");</code>
15	<code>int smallSize = scanner.hasNextInt() ?</code>
16	<code>scanner.nextInt() : 10;</code>
17	<code>int[] originalSmall = new int[smallSize];</code>
18	<code>for (int i = 0; i < smallSize; i++) {</code>
19	<code>originalSmall[i] = rand.nextInt(100); // Random</code>
20	<code>0-99</code>
21	<code>} System.out.println("Array random awal untuk pengujian</code>
22	<code>individu: " + Arrays.toString(originalSmall));</code>
23	
24	<code>while (running) {</code>
25	<code>System.out.println("\nMenu:");</code>
26	<code>System.out.println("1. Jalankan Shell Sort");</code>
27	<code>System.out.println("2. Jalankan Merge Sort");</code>
28	<code>System.out.println("3. Jalankan Quick Sort");</code>
29	<code>System.out.println("4. Bandingkan Kecepatan Semua</code>
30	<code>Algoritma");</code>
31	<code>System.out.println("5. Keluar");</code>
32	<code>System.out.print("Pilih opsi: ");</code>
33	<code>int choice = scanner.nextInt();</code>
34	
35	<code>switch (choice) {</code>
36	<code>case 1:</code>
37	<code>int[] arr1 = Arrays.copyOf(originalSmall,</code>

```

38         runSort("Shell Sort", arr1, (arr,
39 verbose) -> Shell.shellSort(arr, verbose));
40         break;
41         case 2:
42             int[] arr2 = Arrays.copyOf(originalSmall,
43 smallSize);
44             runSort("Merge Sort", arr2, (arr,
45 verbose) -> Merge.mergeSort(arr, verbose));
46             break;
47             case 3:
48                 int[] arr3 = Arrays.copyOf(originalSmall,
49 smallSize);
50                 runSort("Quick Sort", arr3, (arr,
51 verbose) -> Quick.quickSort(arr, 0, arr.length - 1,
52 verbose));
53                 break;
54                 case 4:
55                     compareAll();
56                     break;
57                 case 5:
58                     running = false;
59                     break;
60                 default:
61                     System.out.println("Opsi tidak valid.");
62             }
63         }
64         scanner.close();
65     }
66
67     @FunctionalInterface
68     interface Sorter {
69         void sort(int[] arr, boolean verbose);
70     }
71
72     private static void runSort(String name, int[] arr,
73 Sorter sorter) {
74         System.out.println("\n" + name + ":");
75         System.out.println("Sebelum: " +
76 Arrays.toString(arr));
77         sorter.sort(arr, true);
78         System.out.println("Sesudah: " +
79 Arrays.toString(arr));
80     }
81
82     private static void compareAll() {
83         int size = 1000; // Ukuran array besar untuk
84 pengukuran waktu
85         Random rand = new Random();
86         int[] original = new int[size];
87         for (int i = 0; i < size; i++) {

```



```

83         original[i] = rand.nextInt(100000);
84     }
85     System.out.println("\nArray random awal untuk
86 perbandingan (ukuran " + size + ", ditampilkan sebagian): "
87 + Arrays.toString(Arrays.copyOf(original, 20)) + "...");
88
89     // Shell Sort
90     int[] arr1 = Arrays.copyOf(original, size);
91     long start = System.nanoTime();
92     Shell.shellSort(arr1, false);
93     long end = System.nanoTime();
94     System.out.println("\nWaktu Shell Sort: " + (end -
95 start) / 1_000_000.0 + " ms");
96
97     // Merge Sort
98     int[] arr2 = Arrays.copyOf(original, size);
99     start = System.nanoTime();
100    Merge.mergeSort(arr2, false);
101    end = System.nanoTime();
102    System.out.println("Waktu Merge Sort: " + (end -
103 start) / 1_000_000.0 + " ms");
104
105    // Quick Sort
106    int[] arr3 = Arrays.copyOf(original, size);
107    start = System.nanoTime();
108    Quick.quickSort(arr3, 0, size - 1, false);
109    end = System.nanoTime();
110    System.out.println("Waktu Quick Sort: " + (end -
111 start) / 1_000_000.0 + " ms");
112    }
113 }

```

Screenshot

1. Merge Sort

Memecah *array* menjadi bagian-bagian kecil, mengurutkannya, lalu menggabungkannya kembali. Pada program ini, data awal yang digunakan adalah [4, 16, 07, 15, 10, 80, 35, 60, 73]. Pertama, array dibagi menjadi dua bagian: kiri [4, 16, 07, 15, 10] dan kanan [80, 35, 60, 73]. Masing-masing bagian dipecah lagi hingga tersisa satu elemen, kemudian digabung kembali dengan membandingkan elemen terkecil dari setiap sisi. Hasil penggabungan bagian kiri menjadi [4, 7, 10, 15, 16], sedangkan bagian kanan menjadi [35, 60, 73, 80]. Kedua bagian tersebut kemudian digabung hingga membentuk urutan akhir: [4, 7, 10, 15, 16, 35, 60, 73, 80].

```
Masukkan ukuran array untuk pengujian individu (default 10): 10
Array random awal untuk pengujian individu: [4, 16, 67, 15, 10, 80, 35, 55, 66, 73]

Menu:
1. Jalankan Shell Sort
2. Jalankan Merge Sort
3. Jalankan Quick Sort
4. Bandingkan Kecepatan Semua Algoritma
5. Keluar
Pilih opsi: 2

Merge Sort:
Sebelum: [4, 16, 67, 15, 10, 80, 35, 55, 66, 73]
Split menjadi left: [4, 16, 67, 15, 10] dan right: [80, 35, 55, 66, 73]
Split menjadi left: [4, 16] dan right: [67, 15, 10]
Split menjadi left: [4] dan right: [16]
Merging left: [4] dan right: [16]
Ambil dari left: 4
Array merging sementara: [4, 16]
Ambil sisa right: 16
Array merging sementara: [4, 16]
Hasil merge: [4, 16]
Split menjadi left: [67] dan right: [15, 10]
Split menjadi left: [15] dan right: [10]
Merging left: [15] dan right: [10]
Ambil dari right: 10
Array merging sementara: [10, 10]
Ambil sisa left: 15
Array merging sementara: [10, 15]
Hasil merge: [10, 15]
Merging left: [67] dan right: [10, 15]
Ambil dari right: 10
Array merging sementara: [10, 15, 10]
Ambil dari right: 15
Array merging sementara: [10, 15, 10]
Ambil sisa left: 67
Array merging sementara: [10, 15, 67]
```

```
Ambil dari left: 4
Array merging sementara: [4, 16, 67, 15, 10]
Ambil dari right: 10
Array merging sementara: [4, 10, 67, 15, 10]
Ambil dari right: 15
Array merging sementara: [4, 10, 15, 15, 10]
Ambil dari left: 16
Array merging sementara: [4, 10, 15, 16, 10]
Ambil sisa right: 67
Array merging sementara: [4, 10, 15, 16, 67]
Hasil merge: [4, 10, 15, 16, 67]
Split menjadi left: [80, 35] dan right: [55, 66, 73]
Split menjadi left: [80] dan right: [35]
Merging left: [80] dan right: [35]
Ambil dari right: 35
Array merging sementara: [35, 35]
Ambil sisa left: 80
Array merging sementara: [35, 80]
Hasil merge: [35, 80]
Split menjadi left: [55] dan right: [66, 73]
Split menjadi left: [66] dan right: [73]
Merging left: [66] dan right: [73]
Ambil dari left: 66
Array merging sementara: [66, 73]
Ambil sisa right: 73
Array merging sementara: [66, 73]
Hasil merge: [66, 73]
Merging left: [55] dan right: [66, 73]
Ambil dari left: 55
Array merging sementara: [55, 66, 73]
Ambil sisa right: 66
Array merging sementara: [55, 66, 73]
Ambil sisa right: 73
Array merging sementara: [55, 66, 73]
Hasil merge: [55, 66, 73]
Merging left: [35, 80] dan right: [55, 66, 73]
Ambil dari left: 35
```

```

Ambil dari left: 35
Array merging sementara: [35, 35, 55, 66, 73]
Ambil dari right: 55
Array merging sementara: [35, 55, 55, 66, 73]
Ambil dari right: 66
Array merging sementara: [35, 55, 66, 66, 73]
Ambil dari right: 73
Array merging sementara: [35, 55, 66, 73, 73]
Ambil sisa left: 80
Array merging sementara: [35, 55, 66, 73, 80]
Hasil merge: [35, 55, 66, 73, 80]
Merging left: [4, 10, 15, 16, 67] dan right: [35, 55, 66, 73, 80]
Ambil dari left: 4
Array merging sementara: [4, 16, 67, 15, 10, 80, 35, 55, 66, 73]
Ambil dari left: 10
Array merging sementara: [4, 10, 67, 15, 10, 80, 35, 55, 66, 73]
Ambil dari left: 15
Array merging sementara: [4, 10, 15, 15, 10, 80, 35, 55, 66, 73]
Ambil dari left: 16
Array merging sementara: [4, 10, 15, 16, 10, 80, 35, 55, 66, 73]
Ambil dari right: 35
Array merging sementara: [4, 10, 15, 16, 35, 80, 35, 55, 66, 73]
Ambil dari right: 55
Array merging sementara: [4, 10, 15, 16, 35, 55, 35, 55, 66, 73]
Ambil dari right: 66
Array merging sementara: [4, 10, 15, 16, 35, 55, 66, 55, 66, 73]
Ambil dari left: 67
Array merging sementara: [4, 10, 15, 16, 35, 55, 66, 67, 66, 73]
Ambil sisa right: 73
Array merging sementara: [4, 10, 15, 16, 35, 55, 66, 67, 73, 73]
Ambil sisa right: 80
Array merging sementara: [4, 10, 15, 16, 35, 55, 66, 67, 73, 80]
Hasil merge: [4, 10, 15, 16, 35, 55, 66, 67, 73, 80]
Sesudah: [4, 10, 15, 16, 35, 55, 66, 67, 73, 80]

```

2. Quick Sort

Membagi data menjadi dua bagian berdasarkan elemen yang disebut *pivot*. Semua elemen yang lebih kecil dari *pivot* ditempatkan di sebelah kiri, sedangkan yang lebih besar di sebelah kanan. Proses ini dilakukan secara rekursif hingga seluruh elemen berada dalam urutan yang benar. Pada data awal [4, 16, 07, 15, 10, 80, 35, 60, 73], *pivot* pertama yang dipilih adalah 73. Setelah proses pembagian dan pertukaran elemen, *array* terbagi menjadi dua bagian: kiri [4, 16, 07, 15, 10, 35, 60] dan kanan [80]. Proses yang sama kemudian diulang pada bagian kiri hingga seluruh elemen tersusun dengan benar. Jadi hasil akhir pengurutan dengan *Quick Sort* adalah: [4, 10, 15, 16, 35, 55, 60, 67, 73, 80].

Menu:

1. Jalankan Shell Sort
2. Jalankan Merge Sort
3. Jalankan Quick Sort
4. Bandingkan Kecepatan Semua Algoritma
5. Keluar

Pilih opsi: 3

Quick Sort:

Sebelum: [4, 16, 67, 15, 10, 80, 35, 55, 66, 73]

Pivot: 73 (indeks 9)

Swap: 4 dengan 4 (indeks 0 dan 0)

Array sementara: [4, 16, 67, 15, 10, 80, 35, 55, 66, 73]

Swap: 16 dengan 16 (indeks 1 dan 1)

Array sementara: [4, 16, 67, 15, 10, 80, 35, 55, 66, 73]

Swap: 67 dengan 67 (indeks 2 dan 2)

Array sementara: [4, 16, 67, 15, 10, 80, 35, 55, 66, 73]

Swap: 15 dengan 15 (indeks 3 dan 3)

Array sementara: [4, 16, 67, 15, 10, 80, 35, 55, 66, 73]

Swap: 10 dengan 10 (indeks 4 dan 4)

Array sementara: [4, 16, 67, 15, 10, 80, 35, 55, 66, 73]

Swap: 80 dengan 35 (indeks 5 dan 6)

Array sementara: [4, 16, 67, 15, 10, 35, 80, 55, 66, 73]

Swap: 80 dengan 55 (indeks 6 dan 7)

Array sementara: [4, 16, 67, 15, 10, 35, 55, 80, 66, 73]

Swap: 80 dengan 66 (indeks 7 dan 8)

Array sementara: [4, 16, 67, 15, 10, 35, 55, 66, 80, 73]

Swap pivot: 80 dengan 73 (indeks 8 dan 9)

Array setelah swap pivot: [4, 16, 67, 15, 10, 35, 55, 66, 73, 80]

Setelah partition (pivot di 8): [4, 16, 67, 15, 10, 35, 55, 66, 73, 80]

Pivot: 66 (indeks 7)

Swap: 4 dengan 4 (indeks 0 dan 0)

Array sementara: [4, 16, 67, 15, 10, 35, 55, 66, 73, 80]

Swap: 16 dengan 16 (indeks 1 dan 1)

Array sementara: [4, 16, 67, 15, 10, 35, 55, 66, 73, 80]

Swap: 67 dengan 15 (indeks 2 dan 3)

```

Array sementara: [4, 16, 15, 67, 10, 35, 55, 66, 73, 80]
Swap: 67 dengan 10 (indeks 3 dan 4)
Array sementara: [4, 16, 15, 10, 67, 35, 55, 66, 73, 80]
Swap: 67 dengan 35 (indeks 4 dan 5)
Array sementara: [4, 16, 15, 10, 35, 67, 55, 66, 73, 80]
Swap: 67 dengan 55 (indeks 5 dan 6)
Array sementara: [4, 16, 15, 10, 35, 55, 67, 66, 73, 80]
Swap pivot: 67 dengan 66 (indeks 6 dan 7)
Array setelah swap pivot: [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Setelah partition (pivot di 6): [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Pivot: 55 (indeks 5)
Swap: 4 dengan 4 (indeks 0 dan 0)
Array sementara: [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Swap: 16 dengan 16 (indeks 1 dan 1)
Array sementara: [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Swap: 15 dengan 15 (indeks 2 dan 2)
Array sementara: [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Swap: 10 dengan 10 (indeks 3 dan 3)
Array sementara: [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Swap: 35 dengan 35 (indeks 4 dan 4)
Array sementara: [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Swap pivot: 55 dengan 55 (indeks 5 dan 5)
Array setelah swap pivot: [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Setelah partition (pivot di 5): [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Pivot: 35 (indeks 4)
Swap: 4 dengan 4 (indeks 0 dan 0)
Array sementara: [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Swap: 16 dengan 16 (indeks 1 dan 1)
Array sementara: [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Swap: 15 dengan 15 (indeks 2 dan 2)
Array sementara: [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Swap: 10 dengan 10 (indeks 3 dan 3)
Array sementara: [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Swap pivot: 35 dengan 35 (indeks 4 dan 4)
Array setelah swap pivot: [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]

```

```

Array setelah swap pivot: [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Setelah partition (pivot di 5): [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Pivot: 35 (indeks 4)
Swap: 4 dengan 4 (indeks 0 dan 0)
Array sementara: [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Swap: 16 dengan 16 (indeks 1 dan 1)
Array sementara: [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Swap: 15 dengan 15 (indeks 2 dan 2)
Array sementara: [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Swap: 10 dengan 10 (indeks 3 dan 3)
Array sementara: [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Swap pivot: 35 dengan 35 (indeks 4 dan 4)
Array setelah swap pivot: [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Setelah partition (pivot di 4): [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Pivot: 10 (indeks 3)
Swap: 4 dengan 4 (indeks 0 dan 0)
Array sementara: [4, 16, 15, 10, 35, 55, 66, 67, 73, 80]
Swap pivot: 16 dengan 10 (indeks 1 dan 3)
Array setelah swap pivot: [4, 10, 15, 16, 35, 55, 66, 67, 73, 80]
Setelah partition (pivot di 1): [4, 10, 15, 16, 35, 55, 66, 67, 73, 80]
Pivot: 16 (indeks 3)
Swap: 15 dengan 15 (indeks 2 dan 2)
Array sementara: [4, 10, 15, 16, 35, 55, 66, 67, 73, 80]
Swap pivot: 16 dengan 16 (indeks 3 dan 3)
Array setelah swap pivot: [4, 10, 15, 16, 35, 55, 66, 67, 73, 80]
Setelah partition (pivot di 3): [4, 10, 15, 16, 35, 55, 66, 67, 73, 80]
Sesudah: [4, 10, 15, 16, 35, 55, 66, 67, 73, 80]

```

3. *Shell Sort*

Shell sort merupakan pengembangan dari *Insertion Sort* yang menggunakan konsep jarak atau gap antar elemen untuk mempercepat proses pengurutan. Data awal yang digunakan adalah [4, 16, 67, 15, 10, 80, 35, 55, 66, 73]. Proses dimulai dengan gap = 5, di mana elemen yang berjarak lima posisi dibandingkan dan ditukar jika tidak berurutan. Setelah itu, gap dikurangi menjadi 2, dan pertukaran dilanjutkan untuk memperbaiki posisi elemen. Pada tahap akhir, gap menjadi 1, sehingga algoritma bekerja seperti *Insertion Sort* untuk menyempurnakan hasil. Setelah semua langkah selesai, maka diperoleh hasil akhir pengurutan: [4, 10, 15, 16, 35, 55, 66, 67, 73, 80]. *Shell Sort* lebih cepat dibandingkan *Insertion Sort* karena mampu memindahkan elemen ke posisi yang lebih tepat sejak awal proses pengurutan.

Menu:

1. Jalankan Shell Sort
2. Jalankan Merge Sort
3. Jalankan Quick Sort
4. Bandingkan Kecepatan Semua Algoritma
5. Keluar

Pilih opsi: 1

Shell Sort:

Sebelum: [4, 16, 67, 15, 10, 80, 35, 55, 66, 73]

Gap: 5

Swap: 67 dengan 55

Array sementara: [4, 16, 67, 15, 10, 80, 35, 67, 66, 73]

Insert 55 ke posisi 2

Array setelah insert: [4, 16, 55, 15, 10, 80, 35, 67, 66, 73]

Gap: 2

Swap: 16 dengan 15

Array sementara: [4, 16, 55, 16, 10, 80, 35, 67, 66, 73]

Insert 15 ke posisi 1

Array setelah insert: [4, 15, 55, 16, 10, 80, 35, 67, 66, 73]

Swap: 55 dengan 10

Array sementara: [4, 15, 55, 16, 55, 80, 35, 67, 66, 73]

Insert 10 ke posisi 2

Array setelah insert: [4, 15, 10, 16, 55, 80, 35, 67, 66, 73]

Swap: 55 dengan 35

Array sementara: [4, 15, 10, 16, 55, 80, 55, 67, 66, 73]

Insert 35 ke posisi 4

Array setelah insert: [4, 15, 10, 16, 35, 80, 55, 67, 66, 73]

Swap: 80 dengan 67

Array sementara: [4, 15, 10, 16, 35, 80, 55, 80, 66, 73]

Insert 67 ke posisi 5

Array setelah insert: [4, 15, 10, 16, 35, 67, 55, 80, 66, 73]

Swap: 80 dengan 73

Array sementara: [4, 15, 10, 16, 35, 67, 55, 80, 66, 80]

Insert 73 ke posisi 7

Array setelah insert: [4, 15, 10, 16, 35, 67, 55, 73, 66, 80]


```

Array setelah insert: [4, 15, 10, 16, 35, 67, 55, 73, 66, 80]
Gap: 1
Swap: 15 dengan 10
Array sementara: [4, 15, 15, 16, 35, 67, 55, 73, 66, 80]
Insert 10 ke posisi 1
Array setelah insert: [4, 10, 15, 16, 35, 67, 55, 73, 66, 80]
Swap: 67 dengan 55
Array sementara: [4, 10, 15, 16, 35, 67, 67, 73, 66, 80]
Insert 55 ke posisi 5
Array setelah insert: [4, 10, 15, 16, 35, 55, 67, 73, 66, 80]
Swap: 73 dengan 66
Array sementara: [4, 10, 15, 16, 35, 55, 67, 73, 73, 80]
Swap: 67 dengan 73
Array sementara: [4, 10, 15, 16, 35, 55, 67, 67, 73, 80]
Insert 66 ke posisi 6
Array setelah insert: [4, 10, 15, 16, 35, 55, 66, 67, 73, 80]
Sesudah: [4, 10, 15, 16, 35, 55, 66, 67, 73, 80]

```

4. Perbandingan Kecepatan Antar Algoritma

Perbandingan kecepatan antara *Quick Sort* dan *Shell Sort* ini terjadi karena karakteristik dan cara kerja masing-masing algoritma sorting berbeda. *Quick Sort* lebih unggul pada data besar karena proses partisi dan rekursinya sangat efisien serta memiliki kompleksitas waktu rata-rata. Sedangkan, *Shell Sort* lebih unggul pada data kecil karena prosesnya sederhana, tanpa pembagian atau pemanggilan fungsi rekursif, sehingga lebih efisien untuk jumlah elemen sedikit.

```

Menu:
1. Jalankan Shell Sort
2. Jalankan Merge Sort
3. Jalankan Quick Sort
4. Bandingkan Kecepatan Semua Algoritma
5. Keluar
Pilih opsi: 4

Array random awal untuk perbandingan (ukuran 1000, ditampilkan sebagian): [44076, 59594, 93754, 46434, 94303, 34927, 20852, 56805, 75571, 20061, 67424, 90597, 28987, 69027, 30633, 42016, 61007, 21738, 18111, 63527]...

Waktu Shell Sort: 3.0896 ms
Waktu Merge Sort: 2.4465 ms
Waktu Quick Sort: 1.6856 ms

```

```

Waktu Shell Sort: 3.0896 ms
Waktu Merge Sort: 2.4465 ms
Waktu Quick Sort: 1.6856 ms

```

Array random awal untuk perbandingan (ukuran 10, ditampilkan sebagian): [80649, 25947, 70604, 67573, 32323, 39250, 22359, 23926, 84502, 7081, 0, 0, 0, 0, 0, 0, 0, 0, 0]...

Waktu Shell Sort: 0.0046 ms
Waktu Merge Sort: 0.0101 ms
Waktu Quick Sort: 0.0052 ms

Waktu Shell Sort: 0.0046 ms
Waktu Merge Sort: 0.0101 ms
Waktu Quick Sort: 0.0052 ms

Refrensi

<https://id.scribd.com/document/805616692/Materi-tentang-Shell-sort-Quick-sort-Merge-sort>



Repository

https://github.com/Thisyath/Strukdat-smt-3/tree/master/src/Tugas_3